Unity Portfolio

최진우

Email: devjinwoo0915@gmail.com

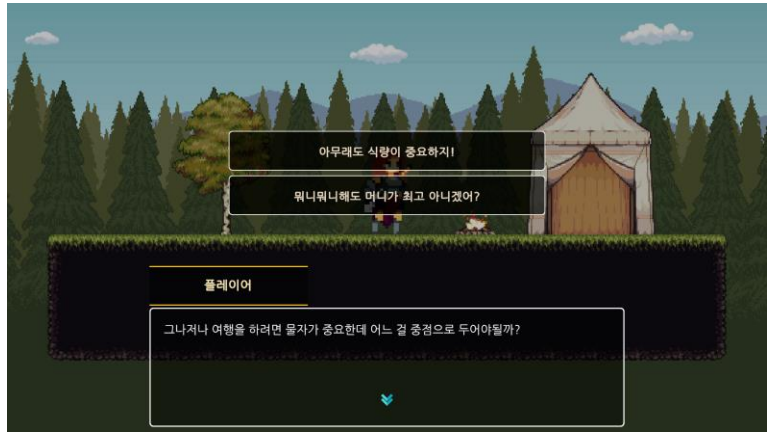
Phone: 010-3953-7961

Restart Adventure

- **개발 기간**: 1월 19일 ~ 2월 23일 (약 1달)
- **개발 인원**: 1인 개발
- **게임 엔진**: Unity 6
- **프로젝트 요약**: 창작 2D 게임으로 캐릭터를 육성하고 자동 전투를 통해 적과 싸우는 게임.
- **GitHub**: https://github.com/jinjinchoi/ProjectR
- **시연영상**: https://youtu.be/ODD-MKS-UR8

게임 소개

- **캐릭터 육성 시스템.**
  - 우마무스메를 참고하여 만들었으며 캐릭터의 능력치를 강화하면 하루가 종료되는 시스템을 구현하였습니다.
  - 육성 시 랜덤하게 스킬 포인트를 얻고 스킬을 습득할 수 있습니다.
- **전투**
  - 전투 이벤트 발생 시 배틀 Scene에 입장하며 이때 플레이어와 에너미는 AI 로직에 따라 자동으로 전투를 합니다.
- **클리어 목표**
  - 모든 날짜가 지날 때까지 캐릭터가 생존하면 게임에서 승리합니다.



핵심 기능 소개

Ability System Component

- **Ability System Component (ASC)**
  - 캐릭터의 스킬과 능력치를 관리하는 클래스로 스킬을 보관하고 실행시킵니다.
  - 능력치의 경우 Attribute Set이라는 별도의 클래스에서 관리하며 해당 클래스는 ASC에서 생성하고 저장합니다.

- **ASC 구현 이유**
  - 캐릭터 클래스에서 스킬이나 능력치를 담당하면 책임이 너무 과도해진다고 생각하였고 또한 스킬이나 능력치가 캐릭터에게 종속되지 않기를 원하여 별도의 관리 클래스를 만들었습니다.
  - 실제로 전투와 달리 육성에는 캐릭터에게 대부분의 기능이 필요 없지만 능력치와 같은 정보는 알아야 하는데 이때 기능이 분리되어 있어 육성 중 사용하는 캐릭터는 일부 기능만 활성화한 채로 구현할 수 있었습니다.

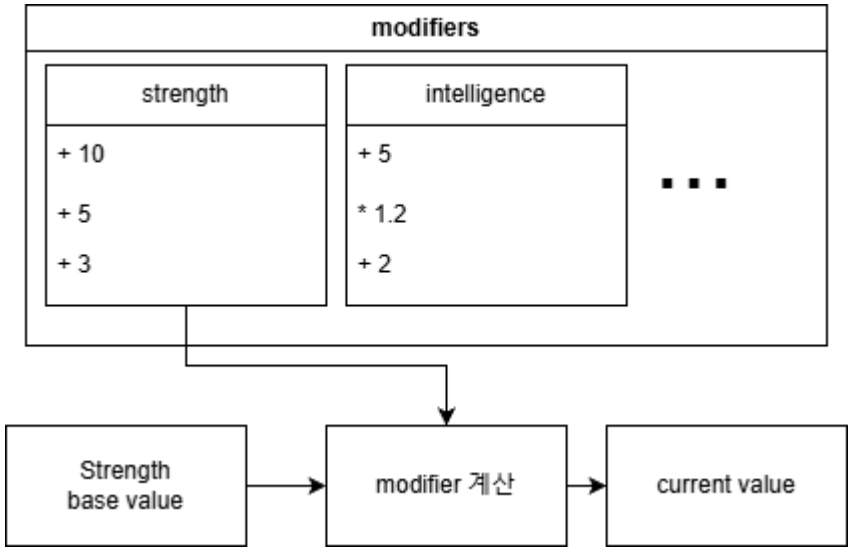
```
/*
 * abilities :현재 보유하고 있는 ability list,
 * activeAbilitySpecs :실행중인 ability map
 * waitingAbilitiesByAnimEvent: anim event를 기다리고 있는 ability map
 */
private readonly List<AbilitySpec> abilities = new(); // 최적화 위해서는 S
private readonly Dictionary<EAbilityId, AbilitySpec> activatedAbilitySpe
private readonly Dictionary<EAnimationEventType, List<WaitingAbilityEntr

private AttributeSet attributeSet;
private IAbilityOwner owner;
```

- ASC에서 관리중인 스킬들과 Attribute Set

능력치

- **Attribute Set**
  - 캐릭터의 능력치(Attribute)를 저장하는 클래스입니다.
  - 능력치는 Base Value에 Modifier를 더하여 값이 정해집니다.
  - Github Link: Attribute 소개

- **Modifier**
  - Modifier는 능력치에 적용하는 변화 값을 정의하는 구조체로 ID를 담은 핸들을 통해 각각의 Modifier를 구분합니다.
  - 단순 Set함수 사용 방식은 결과값만 남기 때문에 여러 효과가 중첩될 때 출처를 추적하거나 특정 효과만 제거하는 것이 어려운 문제가 있었습니다.
  - Modifier를 사용하여 각 효과가 개별 데이터로 남아 중첩 상태를 명확히 파악할 수 있으며, ID를 통해 특정 효과만 선택적으로 제거할 수 있어 이 문제를 해결할 수 있게 하였습니다.
  - Github Link: Modifier 적용 로직



- Modifier 계산 다이어그램으로 Base Value에 Modifier에 있는 값을 더하여 Current value를 설정합니다.

2차 능력치

- **2차 능력치**
  - 1차 능력치는 힘, 민첩 같은 플레이어가 강화가 가능한 것을, 2차 능력치는 물리 공격력처럼 다른 능력치에 따라 변하는 능력치를 말합니다.
  - `AttributeCalculator` 클래스를 만들어 해당 클래스에 다양한 계산식을 통해 2차 능력치를 설정할 수 있도록 하였습니다.
  - Github Link: [2차 Attribute 계산 로직]

- **전략 패턴**
  - 2차 능력치를 설정할 때 분기문으로 구분하면 능력치가 추가될 때마다 작업이 늘어나 가독성과 유지보수성을 떨어뜨릴 수 있습니다.
  - 이를 방지하기 능력치별 계산 클래스를 Map으로 저장한 뒤 필요한 것을 가져와 사용하는 전략 패턴을 활용하여 동일한 함수를 사용해도 서로 다른 능력치를 계산할 수 있게 하였습니다.
  - Github Link: [전략패턴 사용 로직]

```
public class CriticalChanceCalculator : AttributeCalculatorBase
{

    public override EAttributeType TargetAttribute => EAttributeType.CriticalChance;


    public override IReadOnlyList<EAttributeType> Dependencies => new[]
    {
        EAttributeType.Strength,
        EAttributeType.Dexterity
    };


    protected override float CalculateAttribute(AttributeSet attributeSet)
    {
        float str = attributeSet.GetAttributeValue(EAttributeType.Strength);
        float dex = attributeSet.GetAttributeValue(EAttributeType.Dexterity);

        float cirticalChance = 5f + (0.25f * dex) + (0.05f * str);

        return Mathf.Clamp(cirticalChance, 0f, 100f);
    }
}
```

← 의존하고 있는 능력치를 저장

- 크리티컬 확률을 구하고 있는 "Attribute Calculator"

```
// Key: 2차 능력치, value: 계산 클래스
Dictionary<EAttributeType, IAttributeCalculator> calculators
```

- 능력치별 계산 클래스를 저장하는 전략 패턴

2차 능력치

- **Dependency 설정**
  - 처음에는 2차 능력치를 따로 저장하는 것이 아닌 Get함수를 사용할 때 계산 클래스에서 값을 가져와 보여주는 방식을 사용하였습니다.
  - 이렇게 하니 1차 능력치가 바뀌어도 2차 능력치는 변하지 않고 Get함수를 사용해야만 값이 반영된다는 문제가 생겼습니다.
  - 이를 해결하기 위해 1차 능력치와 2차 능력치를 묶은 Map을 통해 의존성을 설정하여 능력치가 변할 때마다 이 Map을 확인해 2차 능력치를 재설정할 수 있도록 하였습니다.
  - Github Link: [dependecy map 생성 로직]

- **DFS(깊이 우선 탐색) 알고리즘을 통한 순환 참조 방지**
  - 능력치가 서로 의존할 경우 무한 루프에 빠지는 문제가 생길 수 있습니다.
  - DFS 알고리즘을 사용하여 순환 유무를 확인하여 이를 방지하였습니다.
  - Github Link: [DFS 알고리즘 로직]

```
// Key: 1차 attribute, Value: key에 의존하고 있는 2차 attributes
Dictionary<EAttributeType, HashSet<EAttributeType>> dependencyMap;
```

- 의존성이 저장된 map

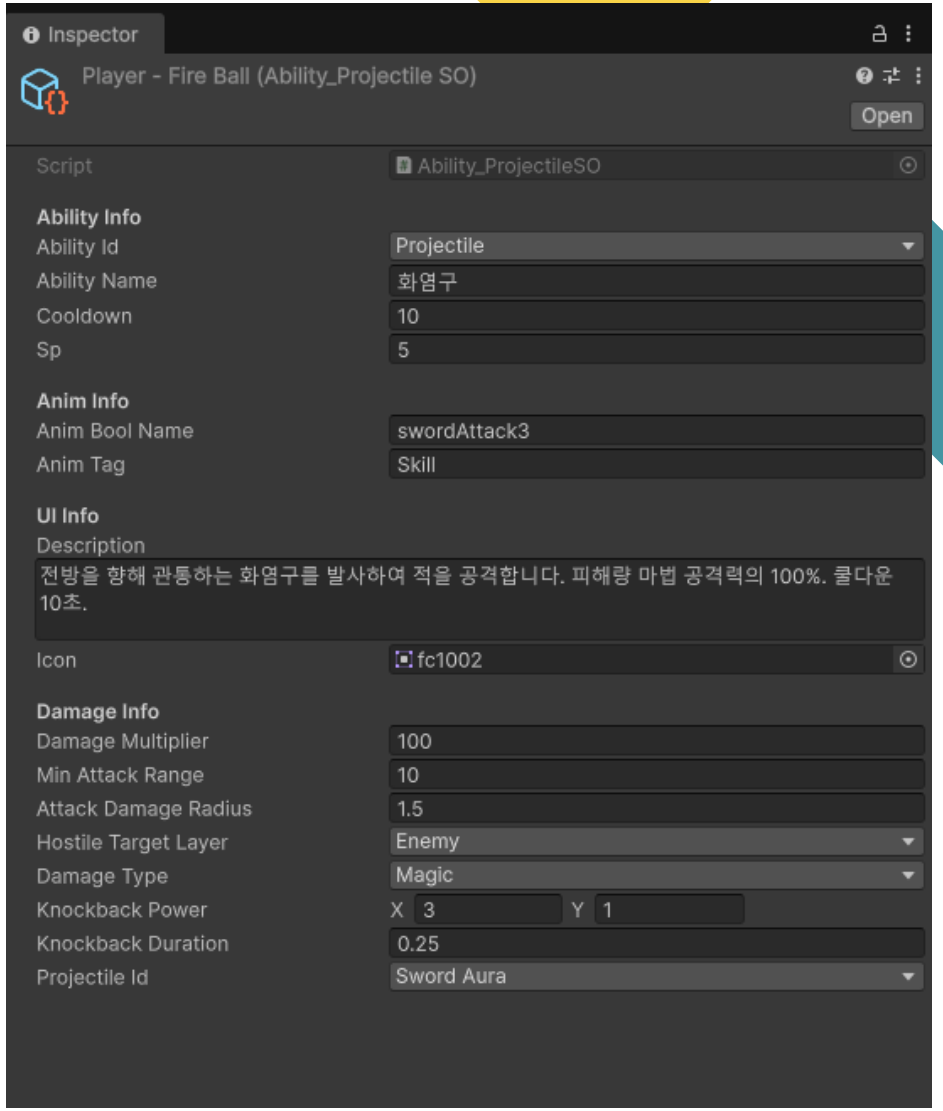
```
// DFS 검사를 통해 그래프에 존재하는 attribute의 자식 노드들을 전부 검사.
2 references
private bool HasCycleDFS(EAttributeType attribute,
    Dictionary<EAttributeType, List<EAttributeType>> graph, Dictionary<EAttributeType, int> visitState)
{
    // 현재 검사중인 attribute 표시
    visitState[attribute] = 1;

    // attribute와 관련된 dependencies 확인
    if (graph.TryGetValue(attribute, out List<EAttributeType> dependencies))
    {
        foreach (var dependentAttribute in dependencies)
        {
            // 현재 검사중인 attribute에 존재하는 dependent attribute가 또 dependency를 가지고 있는지 확인.
            if (!graph.ContainsKey(dependentAttribute)) continue;

            // 해당 attribute를 이미 방문했는지 확인
            if (visitState[dependentAttribute] == 1)
                return true; // 방문했으면 순환 중임

            // 아직 방문 안했으면 방문
            if (visitState[dependentAttribute] == 0)
            {
                // 재귀함수로 검사 시작
                if (HasCycleDFS(dependentAttribute, graph, visitState))
                    return true; // 자식 노드들 중 (순환)true가 반환되면 부모에도 전파
            }
        }
    }

    // 모든 자식 노드 확인 후 검사 완료 표시. 공통된 자식을 가질 수 있기 때문에 별도의 표시 통해 재검사 방지.
    visitState[attribute] = 2;
    return false;
}
```

- DFS 검사 로직

스킬

- **스킬 생성 및 사용**
  - ScriptableObject는 스킬의 데이터를 설정하고 인스턴스 생성함수를 통해 자신의 실행 로직 객체를 생성하여 반환합니다.
  - Ability System Component(ASC)는 이 인스턴스를 공통 추상 클래스로 보관하고 있다가 ID를 전달받으면 이에 맞는 인스턴스를 실행시킵니다.
  - Github Link: [Ability 소개]

- **데이터와 로직 분리**
  - ScriptableObject에서 데이터를 설정하고, 실제 실행 로직은 별도의 인스턴스로 관리하여 데이터와 로직을 분리하였습니다.
  - 이를 통해 동일한 SO를 여러 캐릭터가 공유하더라도 각자 독립적인 상태를 유지할 수 있게 하였습니다.
  - 팩토리 메서드 패턴을 사용하여 인스턴스를 자식 클래스에서 생성할 수 있게 하여 새로운 스킬이 추가되어도 기존 로직은 수정할 필요가 없어 개방-폐쇄 원칙(OCP)을 준수할 수 있도록 하였습니다.

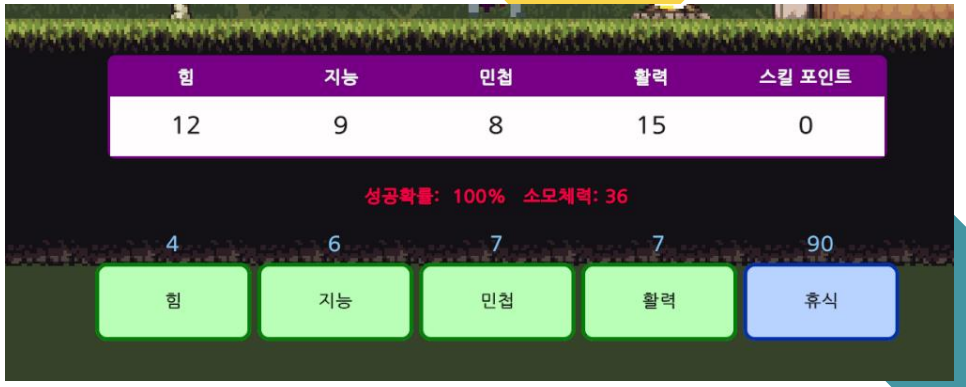


- 스킬을 설정하는 SO의 예시로 이곳에서 애니메이션, 이펙트 등을 설정하여 스킬을 제작할 수 있습니다.

캐릭터 강화

- **능력치 강화**
  - 능력치 강화는 체력에 따라 성공 확률이 정해지며 강화 수치는 랜덤으로 결정됩니다.
  - 강화 수치를 랜덤으로만 설정하면 로드를 할 때마다 값이 변할 것이고 이를 방지 하기위해 별도의 계산 클래스를 만들어 강화 수치를 저장하도록 하였습니다.
  - Github Link: [Attribute 강화 로직]

- **스킬 습득**
  - 캐릭터를 강화하면 일정 수치의 스킬 포인트를 획득하며 이를 통해 스킬을 습득합니다.
  - 스킬의 로직과 관련된 정보는 전투에서만 필요하고 육성에는 필요가 없기에 ID만 Hash Set에 보관하고 전투에 들어가면 이를 확인하여 해당하는 스킬의 인스턴스를 생성하도록 하였습니다.
  - 동일한 스킬은 하나만 습득할 수 있으므로 Set을 사용했으며 UI에서는 전체 스킬의 ID랑 습득한 ID를 비교하여 습득 여부를 보여줄 수 있게 하였습니다.
  - Github Link: [Ability 습득 로직]



- Attribute 강화 화면

- 스킬창

대미지 시스템

- **Damage Calculator**
  - 대미지는 단순히 계수로만 정해지는 것이 아닌 능력치의 영향을 받기에 계산 클래스를 따로 만들어 공격력이나 방어력 같은 수치를 쉽게 계산할 수 있도록 하였습니다.
  - Github Link: [대미지 적용 로직 소개]

- **DamageInfo 구조체**
  - 대미지를 구현할 때 수치만 전달하니 넉백과 같은 반응을 구현하기 위한 정보가 부족하다는 문제가 발생했습니다.
  - 수치뿐만 아니라 Instigator의 위치나 넉백 파워 등이 담긴 구조체를 보내 다양한 반응을 할 수 있게 하여 이 문제를 해결했습니다.
  - 이러한 대미지 정보는 전투 중 자주 생성하고 소멸하는 데이터이므로 GC 발생을 줄이기 위하여 구조체를 사용하였습니다.

```
public struct FDamageInfo
{
    public IAbilityOwner Instigator;
    public Transform DamageSource;
    public float OriginalDamage;
    public EDamageType DamageType;
    public bool IsCritical;
    public Vector2 KnockbackPower;
    public float KnockbackDuration;
```

- 대미지 정보를 저장하는 구조체

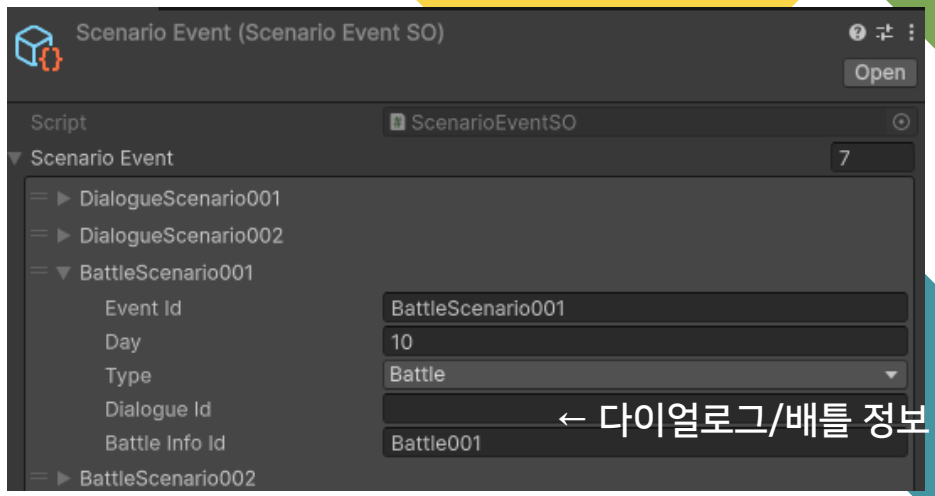
```
FDamageInfo damageInfo = DamageCalculator.CalculateOutgoingDamage(c

foreach (var hit in hits)
{
    if (hit.TryGetComponent<IDamageable>(out var damageable))
    {
        damageable.TakeDamage(damageInfo);
    }
}
```

- 대미지를 입히는 예시로 인터페이스를 사용하여 대미지 정보를 보내 피해 계산

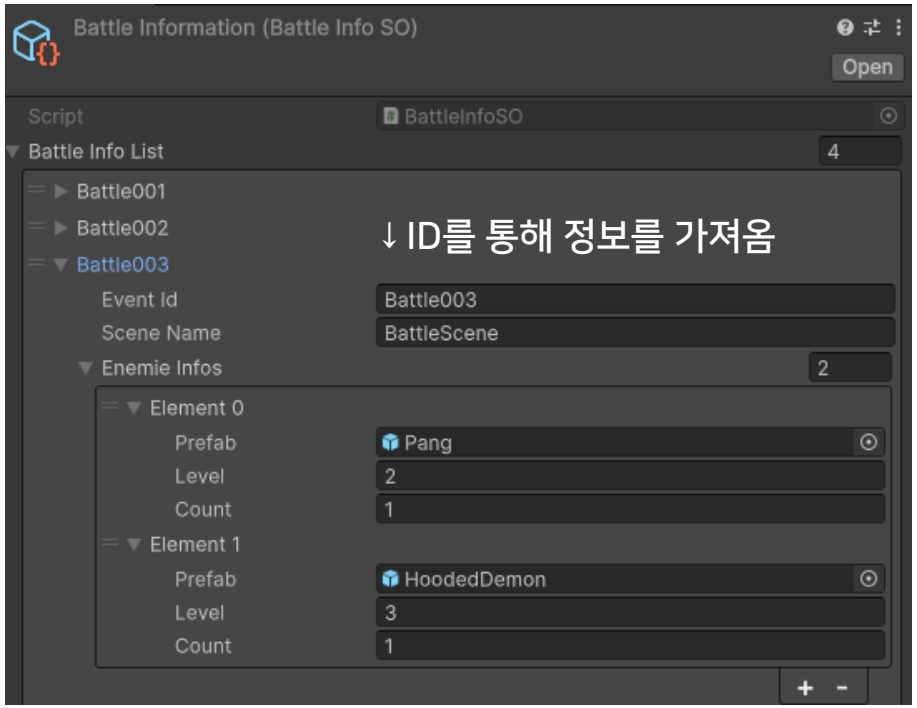
이벤트, 다이얼로그 시스템

- **이벤트 & 다이얼로그**
  - 특정 날짜나 랜덤으로 이벤트가 발생하여 다이얼로그나 전투를 진행하도록 하였습니다.
  - 이 시스템에서 가장 중요하게 생각한 점은 서로 다른 데이터라 할지라도 상관없이 연결할 수 있도록 하는 것이었고 이를 위해 ID를 사용하였습니다.
  - ID를 통해 연결하기 때문에 전혀 다른 시스템이라고 할지라도 연결이 가능해져 확장성이 증가하게 되었습니다.
  - Github Link: 이벤트 시스템 소개 / 다이얼로그 시스템 소개

- **전투 이벤트**
  - 전투에 돌입하게 되면 ID를 통해 에너미에 대한 정보를 가져와 생성하게 됩니다.
  - 에너미 데이터와 전투 데이터를 분리하여 전투 데이터에서 레벨을 통해 에너미의 강함을 설정할 수 있기 때문에 간단하고 재사용 가능한 전투 이벤트를 제작할 수 있습니다.



- 이벤트를 설정하는 SO

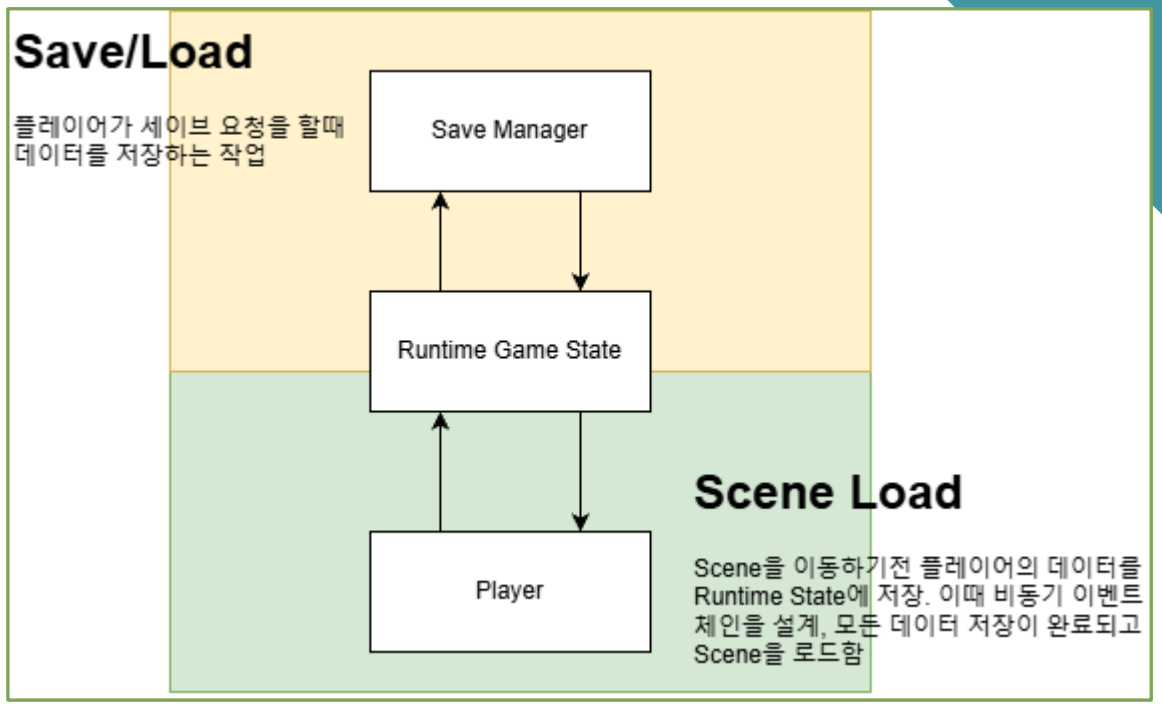


- 전투시 사용할 적의 정보를 설정하는 SO

Save & Load

- **Runtime Game State**
  - Scene을 이동할 때 캐릭터의 데이터를 저장하는 클래스입니다.
  - 캐릭터는 Scene을 이동할 때마다 자신의 Attribute와 습득한 Ability를 이곳에서 가져와 복원합니다

- **Save와 Load 작업에서 Runtime Game State 사용**
  - 처음 세이브, 로드를 구현할 때 캐릭터 클래스가 직접 세이브 매니저에 접근해서 데이터를 복원하였습니다.
  - 그러던 중 Game State가 하는 작업과 세이브 매니저가 하는 작업 간의 경계가 모호하다고 느껴지기 시작하였습니다.
  - 캐릭터 클래스는 Scene이 시작되면 Game State에 접근하기 때문에 이곳에 데이터를 설정하면 데이터 로드가 가능해진다는 사실을 알았습니다.
  - 로드 시 Game State에 데이터를 저장하여 Scene 이동 후 데이터를 불러오도록 하였고, 세이브도 Game State에서 데이터를 가져와 저장하도록 하였습니다.
  - Github Link: [Save System 소개]



AI

- **AI Controller**
  - AI Controller는 타겟의 존재나 스킬 사용 가능 여부 등 캐릭터의 상태를 판단하는 클래스입니다.

- **Finite State Machine (FSM)**
  - FSM을 통하여 State별로 캐릭터의 행동을 정의하였습니다
  - Github Link: AI 소개

- **책임 분리**
  - 처음 로직을 구현할 때 캐릭터 클래스에 접근해서 이동을 시키는 것은 어느 클래스가 해야 하는 것인가에 대한 의문이 생겼습니다.
  - FSM이 행동을 실행한다고 정했기 이동이나 공격도 FSM에서 해야 하는지 헷갈렸고 계속 고민하던 중 이론만 보는 것이 아닌 실제 코드를 보며 역할을 구분하기로 하였습니다.
  - 코드를 보니 AIC가 상태를 결정하기 위해서는 반드시 캐릭터에 접근해야 하고 FSM은 State를 바꾸기 위해 반드시 AIC에 접근해야 한다는 사실을 알게 되었습니다.
  - 이에 캐릭터에 접근하는 작업은 AIC에서 하고, FSM은 AIC에 이동 요청을 하도록 하여 단일책임 원칙을 이루어냈습니다.

```
11 references
public override void Update()
{
    base.Update();

    if (!aiController.HasTarget)
    {
        stateMachine.ChangeState(aiController.IdleState);
        return;
    }

    if (aiController.CanEnterSkillState(aiController.PendingAbilityId))
    {
        stateMachine.ChangeState(aiController.SkillState);
        return;
    }

    if (aiController.CanEnterAttackState())
    {
        stateMachine.ChangeState(aiController.AttackState);
        return;
    }

    aiController.MovoToTarget();
}
```

- Combat State의 로직 중 일부로 state는 AIC에 현재 캐릭터의 상태를 물어 특정 State로 이동할지 아니면 행동을 계속할지를 결정합니다.

AI

- **랜덤 Ability 사용**
  - AIC는 코루틴을 사용하여 일정 간격으로 ASC에서 랜덤으로 스킬의 id를 가져와 저장하고 State는 이를 확인하여 스킬 발동을 요청합니다.

- **Fisher–Yates Shuffle Algorithm**
  - 랜덤으로 스킬을 사용하기 위해 랜덤 인덱스를 이용하면 쿨다운 중이거나 실행 중인 스킬의 ID를 가져올 수도 있습니다.
  - 그렇기에 셔플 알고리즘을 사용하여 스킬 배열의 인덱스를 섞은 뒤 차례대로 확인하여 발동 가능한 스킬의 Id를 가져오는 방식을 구현하였습니다.
  - 배열 전체를 섞는 것이 아닌 Integer로만 이루어진 배열을 섞어 최적화를 하였습니다.
  - Github Link: [전체 Ability 발동 로직]

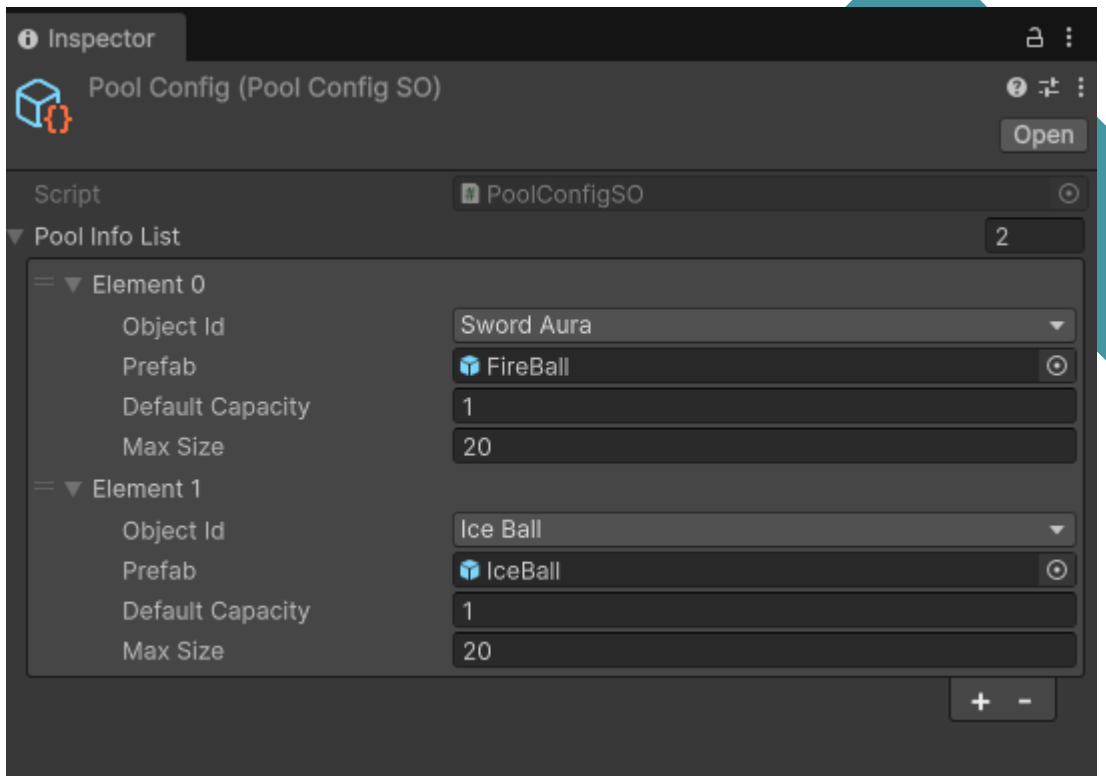
```
// Fisher-Yates Shuffle Algorithm (무작위 순열 생성 알고리즘)
// 배열의 마지막 요소부터 시작  // 배열의 첫번째 요소에 도착할 때까지 반복
for (int i = indices.Count - 1; i > 0; i--)
{
    // 0부터 현재 인덱스(i) 사이에서 무작위 정수 j를 선택
    int j = UnityEngine.Random.Range(0, i + 1);
    // i와 j의 위치를 서로 변경
    (indices[i], indices[j]) = (indices[j], indices[i]);
}

// 순서대로 검사
foreach (int index in indices)
{
    var spec = abilities[index];
    if (spec.ability.CanActivate(spec, this))
    {
        return spec.abilityData.abilityId;
    }
}
```

- ASC에서 Ability Id를 반환하는 코드 중 일부.

Object Pooling

- **Effect Pool**
  - 오브젝트 풀은 유니티가 제공해 주는 API를 사용하여 구현하였습니다.
  - Effect Pool은 동일한 오브젝트를 저장한 뒤 애니메이션만 바꾸어 서로 다른 이펙트를 주도록 구현한 Pool입니다.

- **일반 Pool**
  - 오브젝트를 하나만 사용하여 이펙트를 구현한 방식은 간단하다는 장점이 있지만 오브젝트별로 설정을 다르게 할 수 없다는 문제가 발생하였습니다.
  - 이를 해결하기 위해 오브젝트별 pool을 만들었고 ID로 구분하여 쉽게 원하는 오브젝트를 찾을 수 있게 하였습니다.
  - Github Link: [오브젝트 풀링 소개]

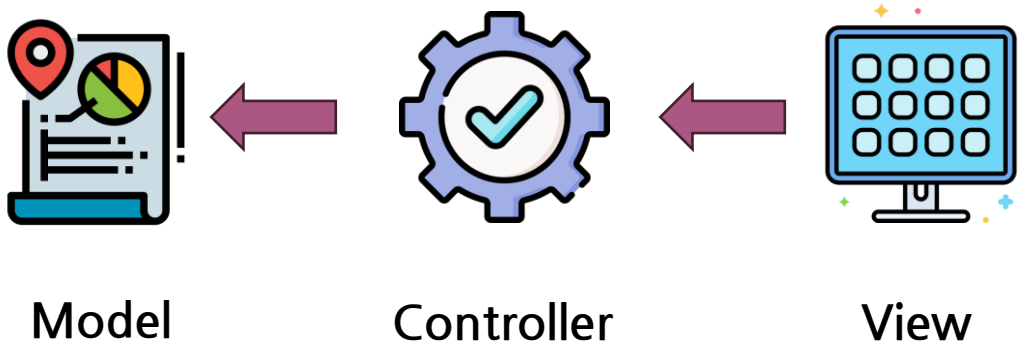


- 일반 Pool에서 사용하는 ScriptableObject로 pool에서 사용할 정보를 설정.

Model-View-Controller Pattern

- **MVC 패턴 구현**
  - 모델은 데이터의 변경을 event를 통해 알립니다. 컨트롤러는 이를 구독하고 다시 UI에 알리며 UI는 이를 수신해 화면에 보여줍니다.
  - MVC 패턴을 사용하여 UI가 데이터에 간섭하는 일을 막아 책임을 분리하고 유지보수가 용이하게 하였습니다.

- **실제 MVC 패턴을 사용하여 도움이 된 사례**
  - 캐릭터를 강화할 때 컨트롤러가 계산 클래스에 접근하고 있었습니다.
  - MVC 패턴 철학에 맞는지를 차치하더라도 세이브 로드 시 강화 수치를 저장하기 어렵다는 문제점이 있었고 이를 해결하기 위해 Game State에서 강화 수치를 보관하기로 하였습니다.
  - 이러한 리팩토링 과정에서 UI의 경우 아무런 코드를 수정할 필요가 없었으며 컨트롤러만 강화 수치에 접근하는 방법만 바꾸면 돼 매우 간단하게 작업이 끝났었습니다.



Unreal Project

Unreal Project

Brand New RPG

- **소개**: 리슨 서버 기반의 멀티플레이 게임으로 기본적인 전투, 인벤토리, 세이브&로드, 퀘스트 시스템 등이 구현된 프로젝트입니다.
- **제작 기간**: 2025. 08. 21 ~ 2025. 11. 21
- **시연 영상**: 유튜브

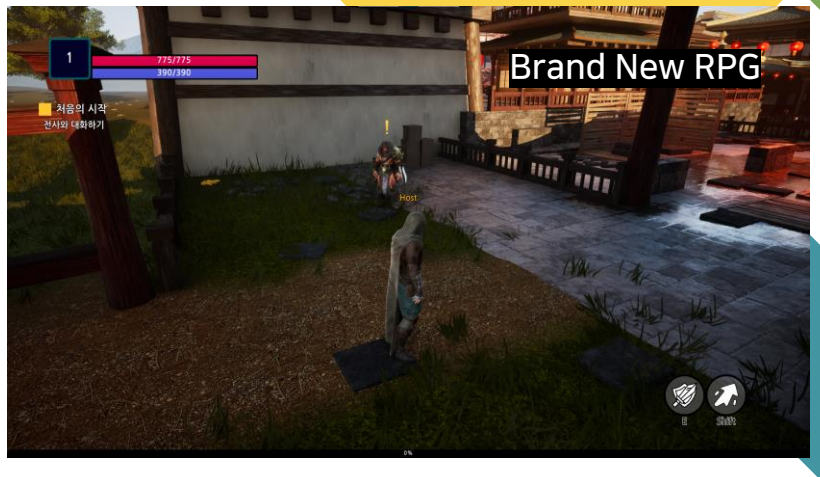
Adventure RPG

- **소개**: 싱글 플레이 게임으로 원신이나 명조와 같이 파티를 구성하고 캐릭터를 전환하는 로직을 담은 프로젝트입니다.
- **제작 기간**: 2025. 03. 27 ~ 2025. 05. 16
- **시연 영상**: 유튜브

Dungeon Exploration

- **소개**: 싱글 플레이 게임으로 던전을 클리어하는 탑다운 뷰 방식의 프로젝트입니다. 네트워크 복제를 설정하여 멀티플레이에서도 로직들이 작동하게 제작되었습니다.
- 제작 기간: 2025. 01. 14 ~ 2025. 02. 09
- **시연 영상**: 유튜브

전체 프로젝트 소개 PDF: 구글 드라이브



감사합니다.